MINOR PROJECT:03

TITLE:DESIGN AND VERIFICATION OF 16×8 SINGLE-PORT AND DUAL-PORT RAM USING VERILOG HDL

SUBMITTED BY:S. GAYATHRI CHANDANA

BATCH: VLSI

RAM 16 × 8 DESIGN REPORT

Single-Port and Dual-Port Synchronous RAM

Based on the provided RAM 16 × 8 Design Specification

1. Introduction

This report presents the design specification and verification requirements for a 16 × 8 synchronous RAM. The design covers both single-port and true dual-port configurations. The memory contains 16 locations, with each location storing 8 bits.

The report follows the supplied learning-oriented RTL specification and retains its stated clocking, interface, operation, collision, and verification requirements.

```
Ln#
 1  module single_port_ram (
 2      input clk,
 3      input rst,
 4      input en,
 5      input we,
 6      input [3:0] addr,
 7      input [7:0] wdata,
 8      output reg [7:0] rdata
 9  );
10
11      reg [7:0] mem [0:15];
12
13      always @(posedge clk) begin
14
15          if (en) begin
16
17              if (we) begin
18                  mem[addr] <= wdata;
19              end
20
21              else begin
22                  rdata <= mem[addr];
23              end
24
25          end
26
27      end
28
29  endmodule
```

Code of single port ram

```
`timescale 1ns/1ps

module tb_single_port_ram;

    reg clk;
    reg rst;
    reg en;
    reg we;
    reg [3:0] addr;
    reg [7:0] wdata;
    wire [7:0] rdata;

    integer i;

    single_port_ram DUT (
        .clk(clk),
        .rst(rst),
        .en(en),
        .we(we),
        .addr(addr),
        .wdata(wdata),
        .rdata(rdata)
    );


    always #5 clk = ~clk;

    initial begin

        clk = 0;
        rst = 0;
        en = 0;
        we = 0;
        addr = 0;
        wdata = 0;
addr = 0;
wdata = 0;



en = 1;
we = 1;
addr = 4'd0;
wdata = 8'h00;

@(posedge clk);
#1;

we = 0;

@(posedge clk);
#1;

if (rdata == 8'h00)
    $display("PASS: Pattern 00");
else
    $display("FAIL: Pattern 00");



we = 1;
addr = 4'd1;
wdata = 8'hFF;

@(posedge clk);
#1;

we = 0;

@(posedge clk);
```

```
        we = 0;

        @(posedge clk);
        #1;

        if (rdata == 8'hFF)
            $display("PASS: Pattern FF");
        else
            $display("FAIL: Pattern FF");



        we = 1;
        addr = 4'd2;
        wdata = 8'hAA;

        @(posedge clk);
        #1;

        we = 0;

        @(posedge clk);
        #1;

        if (rdata == 8'hAA)
            $display("PASS: Pattern AA");
        else
            $display("FAIL: Pattern AA");



        we = 1;
        addr = 4'd3;
        wdata = 8'h55;
```

```
Ln#
102         wdata = 8'h55;
103
104         @(posedge clk);
105         #1;
106
107         we = 0;
108
109         @(posedge clk);
110         #1;
111
112         if (rdata == 8'h55)
113             $display("PASS: Pattern 55");
114         else
115             $display("FAIL: Pattern 55");
116
117
118
119
120         we = 1;
121         addr = 4'd4;
122         wdata = 8'h3C;
123
124         @(posedge clk);
125         #1;
126
127         we = 0;
128
129         @(posedge clk);
130         #1;
131
132         if (rdata == 8'h3C)
133             $display("PASS: Random Pattern 3C");
134         else
135             $display("FAIL: Random Pattern 3C");
136
137
```

```
127         we = 0;
128
129         @(posedge clk);
130         #1;
131
132         if (rdata == 8'h3C)
133             $display("PASS: Random Pattern 3C");
134         else
135             $display("FAIL: Random Pattern 3C");
136
137
138
139         en = 0;
140         we = 0;
141
142         #10;
143
144         $stop;
145
146     end
147
148 endmodule
```

Code of testbench for single port RAM

```
Ln#
 1  module dual_port_ram (
 2      // Port A
 3      input clk_a,
 4      input en_a,
 5      input we_a,
 6      input [3:0] addr_a,
 7      input [7:0] wdata_a,
 8      output reg [7:0] rdata_a,
 9
10      // Port B
11      input clk_b,
12      input en_b,
13      input we_b,
14      input [3:0] addr_b,
15      input [7:0] wdata_b,
16      output reg [7:0] rdata_b
17  );
18
19      // Shared 16 x 8 memory
20      reg [7:0] mem [0:15];
21
22      // Port A
23      always @(posedge clk_a) begin
24          if (en_a) begin
25              if (we_a)
26                  mem[addr_a] <= wdata_a;
27              else
28                  rdata_a <= mem[addr_a];
29          end
30      end
31
32      // Port B
33      always @(posedge clk_b) begin
34          if (en_b) begin
35              if (we_b)
36                  mem[addr_b] <= wdata_b;
```

```
        end
    end

    // Port B
    always @(posedge clk_b) begin
        if (en_b) begin
            if (we_b)
                mem[addr_b] <= wdata_b;
            else
                rdata_b <= mem[addr_b];
        end
    end

endmodule
```

Code for dual port RAM

```
`timescale 1ns/1ps

module tb_dual_port_ram;
    reg clk_a;
    reg en_a;
    reg we_a;
    reg [3:0] addr_a;
    reg [7:0] wdata_a;
    wire [7:0] rdata_a;

    reg clk_b;
    reg en_b;
    reg we_b;
    reg [3:0] addr_b;
    reg [7:0] wdata_b;
    wire [7:0] rdata_b;

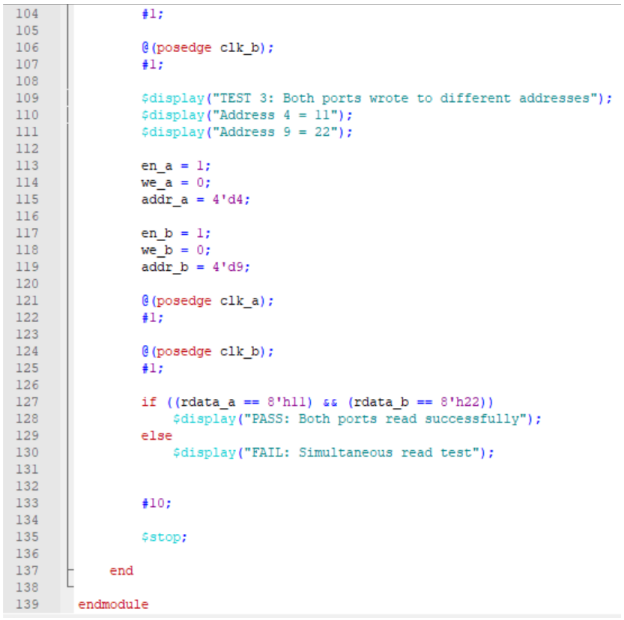
    dual_port_ram DUT (
        .clk_a(clk_a),
        .en_a(en_a),
        .we_a(we_a),
        .addr_a(addr_a),
        .wdata_a(wdata_a),
        .rdata_a(rdata_a),

        .clk_b(clk_b),
        .en_b(en_b),
        .we_b(we_b),
        .addr_b(addr_b),
        .wdata_b(wdata_b),
        .rdata_b(rdata_b)
    );

    always #5 clk_a = ~clk_a;
    always #5 clk_b = ~clk_b;
```

```
Ln#
34      always #5 clk_a = ~clk_a;
35      always #5 clk_b = ~clk_b;
36
37      initial begin
38
39          clk_a = 0;
40          clk_b = 0;
41
42          en_a = 0;
43          we_a = 0;
44          addr_a = 0;
45          wdata_a = 0;
46
47          en_b = 0;
48          we_b = 0;
49          addr_b = 0;
50          wdata_b = 0;
51
52          en_a = 1;
53          we_a = 1;
54          addr_a = 4'd2;
55          wdata_a = 8'h55;
56
57          en_b = 1;
58          we_b = 0;
59          addr_b = 4'd7;
60
61          @(posedge clk_a);
62          #1;
63
64          @(posedge clk_b);
65          #1;
66
67          $display("TEST 1: Port A Write / Port B Read");
68          $display("Port A wrote 55 to address 2");
69          $display("Port B read address 7 = %h", rdata_
```

```
69          $display("Port B read address 7 = %h", rdata_
70
71          en_a = 1;
72          we_a = 0;
73          addr_a = 4'd2;
74
75          en_b = 1;
76          we_b = 1;
77          addr_b = 4'd7;
78          wdata_b = 8'hAA;
79
80          @(posedge clk_a);
81          #1;
82
83          @(posedge clk_b);
84          #1;
85
86          if (rdata_a == 8'h55)
87              $display("PASS: Port A read = %h", rdata_a);
88          else
89              $display("FAIL: Port A read = %h", rdata_a);
90
91          $display("Port B wrote AA to address 7");
92
93          en_a = 1;
94          we_a = 1;
95          addr_a = 4'd4;
96          wdata_a = 8'h11;
97
98          en_b = 1;
99          we_b = 1;
100         addr_b = 4'd9;
101         wdata_b = 8'h22;
102
103         @(posedge clk_a);
104         #1;
```



```
104         #1;
105
106         @(posedge clk_b);
107         #1;
108
109         $display("TEST 3: Both ports wrote to different addresses");
110         $display("Address 4 = 11");
111         $display("Address 9 = 22");
112
113         en_a = 1;
114         we_a = 0;
115         addr_a = 4'd4;
116
117         en_b = 1;
118         we_b = 0;
119         addr_b = 4'd9;
120
121         @(posedge clk_a);
122         #1;
123
124         @(posedge clk_b);
125         #1;
126
127         if ((rdata_a == 8'h11) && (rdata_b == 8'h22))
128             $display("PASS: Both ports read successfully");
129         else
130             $display("FAIL: Simultaneous read test");
131
132
133         #10;
134
135         $stop;
136
137     end
138
139 endmodule
```

code for testbench of dual port ram

2. Common Memory Specification

Parameter	Specification
Memory depth	16 locations
Data width	8 bits per location
Total capacity	16 × 8 = 128 bits = 16 bytes
Address width	4 bits (A[3:0])
Data width	8 bits (D[7:0])
Reset behavior	Memory contents are not reset unless explicitly specified
Clocking	Synchronous with rising edge of clock

3. Single-Port RAM

A single-port RAM has one shared access port. In a given clock cycle, the port performs either a read or a write operation.

3.1 Interface

Signal	Width	Direction	Description
clk	1	Input	Rising-edge clock
rst	1	Input	Synchronous/asynchronous reset; recommended use is to reset control outputs, not RAM contents
en	1	Input	Port enable. Operation occurs only when en = 1
we	1	Input	Write enable. we = 1 writes; we = 0 performs read
addr	4	Input	Memory address, 0 to 15
wdata	8	Input	Data written to selected address

rdata	8	Output	Data read from selected address

3.2 Operation Rules

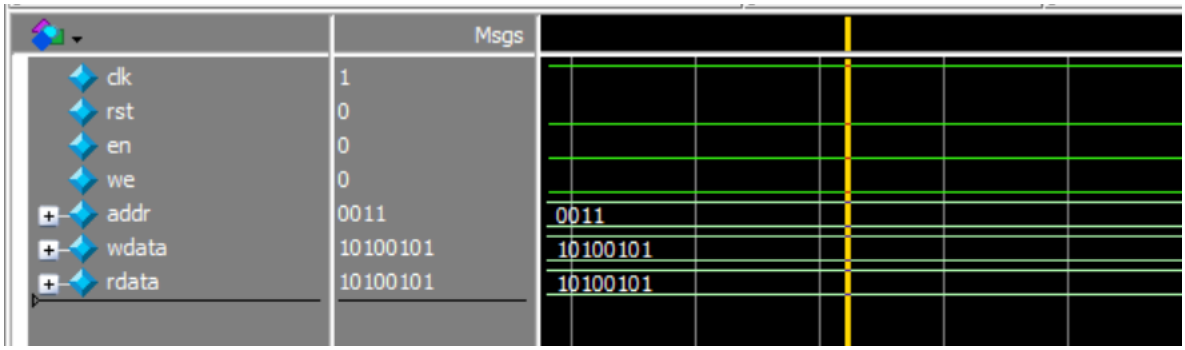
- If en = 0, no memory operation is performed.
- If en = 1 and we = 1, mem[addr] is updated with wdata at the rising clock edge.
- If en = 1 and we = 0, rdata is updated with mem[addr] at the rising clock edge.
- Recommended read behavior is synchronous, so rdata changes on the clock edge.
- Simultaneous read and write are not allowed for the single port because the port supports only one operation per cycle.

```
Ln#
 1  module single_port_ram (
 2      input clk,
 3      input rst,
 4      input en,
 5      input we,
 6      input [3:0] addr,
 7      input [7:0] wdata,
 8      output reg [7:0] rdata
 9  );
10
11      reg [7:0] mem [0:15];
12
13      always @(posedge clk) begin
14
15          if (en) begin
16
17              if (we) begin
18                  mem[addr] <= wdata;
19              end
20
21              else begin
22                  rdata <= mem[addr];
23              end
24
25          end
26
27      end
28
29  endmodule
```

screenshot of the single-port RAM block diagram / RTL

3.3 Example Timing

Cycle	en	we	addr	wdata	Operation	Expected rdata
1	1	1	0011	A5	Write A5 to address 3	Don't care
2	1	0	0011	XX	Read address 3	A5
3	0	X	XXXX	XX	Idle	Holds previous value



screenshot of single-port simulation waveform / timing diagram

4. Dual-Port RAM

A true dual-port RAM provides two independent access ports to the same 16 × 8 memory array. Each port has its own address, enable, write enable, write data, and read data.

4.1 Interface

Signal	Width	Direction	Description
clk_a	1	Input	Clock for Port A
en_a	1	Input	Port A enable
we_a	1	Input	Port A write enable
addr_a	4	Input	Port A address
wdata_a	8	Input	Port A write data
rdata_a	8	Output	Port A read data
clk_b	1	Input	Clock for Port B
en_b	1	Input	Port B enable
we_b	1	Input	Port B write enable
addr_b	4	Input	Port B address
wdata_b	8	Input	Port B write data
rdata_b	8	Output	Port B read data

4.2 Port Operation and Collision Behavior

- Port A and Port B can operate independently.
- A read occurs when en = 1 and we = 0.
- A write occurs when en = 1 and we = 1.
- Both ports may access different addresses in the same cycle.
- If both ports write the same address on the same effective clock edge, the result is a write-write collision and is considered undefined.
- If one port writes an address while the other reads the same address on the same effective clock edge, the read-during-write result must be defined by the implementation.

```
`timescale 1ns/1ps

module tb_dual_port_ram;
    reg clk_a;
    reg en_a;
    reg we_a;
    reg [3:0] addr_a;
    reg [7:0] wdata_a;
    wire [7:0] rdata_a;

    reg clk_b;
    reg en_b;
    reg we_b;
    reg [3:0] addr_b;
    reg [7:0] wdata_b;
    wire [7:0] rdata_b;

    dual_port_ram DUT (
        .clk_a(clk_a),
        .en_a(en_a),
        .we_a(we_a),
        .addr_a(addr_a),
        .wdata_a(wdata_a),
        .rdata_a(rdata_a),

        .clk_b(clk_b),
        .en_b(en_b),
        .we_b(we_b),
        .addr_b(addr_b),
        .wdata_b(wdata_b),
        .rdata_b(rdata_b)
    );

    always #5 clk_a = ~clk_a;
    always #5 clk_b = ~clk_b;
```

```
        end
    end

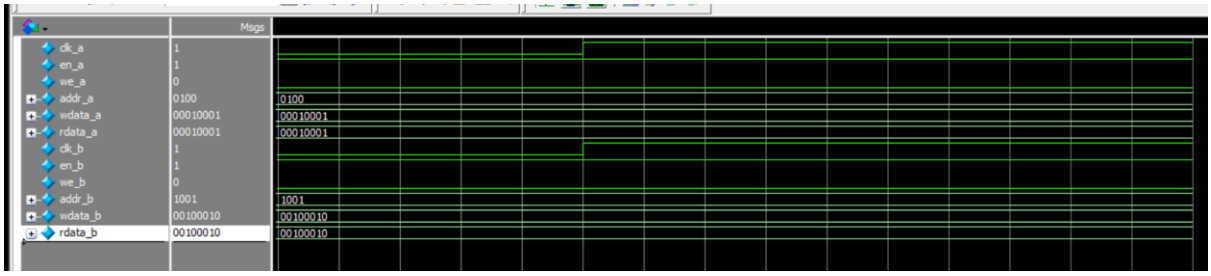
    // Port B
    always @(posedge clk_b) begin
        if (en_b) begin
            if (we_b)
                mem[addr_b] <= wdata_b;
            else
                rdata_b <= mem[addr_b];
        end
    end

endmodule
```

screenshot of the dual-port RAM block diagram / RTL here

4.3 Dual-Port Example

Cycle	Port A	Port B	Expected memory effect
1	Write 55 @ addr 2	Read @ addr 7	addr 2 = 55; Port B returns old addr 7 data
2	Read @ addr 2	Write AA @ addr 7	Port A returns 55; addr 7 = AA
3	Write 11 @ addr 4	Write 22 @ addr 9	addr 4 = 11 and addr 9 = 22



screenshot of dual-port simulation waveform / timing diagram

```
# TEST 1: Port A Write / Port B Read
# Port A wrote 55 to address 2
# Port B read address 7 = xx
# PASS: Port A read = 55
# Port B wrote AA to address 7
# TEST 3: Both ports wrote to different addresses
# Address 4 = 11
# Address 9 = 22
# PASS: Both ports read successfully
# ** Note: $stop    : D:/tb_dual_port_ram.v(164)
#    Time: 86 ns  Iteration: 0  Instance: /tb_dual_port_ram
# Break in Module tb_dual_port_ram at D:/tb_dual_port_ram.v line 164
```

Dual-Port RAM functional verification results

```
VSIM 10> run 200ns
# ------------------------------
# All 16 addresses written.
# ------------------------------
# PASS: Address = 0, Data = 01
# PASS: Address = 1, Data = 02
# PASS: Address = 2, Data = 03
# PASS: Address = 3, Data = 04
```

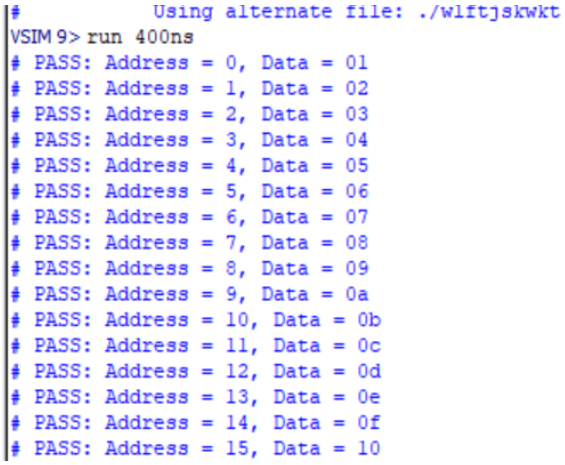
Dual-Port RAM verification for all 16 memory locations

```
# PASS: Address = 4, Data = 05
# PASS: Address = 5, Data = 06
# PASS: Address = 6, Data = 07
# PASS: Address = 7, Data = 08
# PASS: Address = 8, Data = 09
# PASS: Address = 9, Data = 0a
# PASS: Address = 10, Data = 0b
# PASS: Address = 11, Data = 0c
# PASS: Address = 12, Data = 0d
# PASS: Address = 13, Data = 0e
# PASS: Address = 14, Data = 0f
# PASS: Address = 15, Data = 10
# ------------------------------
# Dual-Port all-address verification completed.
# ------------------------------
# ** Note: $stop    : D:/tb_dual_port_all_addresses.v(94)
#    Time: 326 ns  Iteration: 0  Instance: /tb_dual_port_all_addresses
# Break in Module tb_dual_port_all_addresses at D:/tb_dual_port_all_addresses.v line 94
```

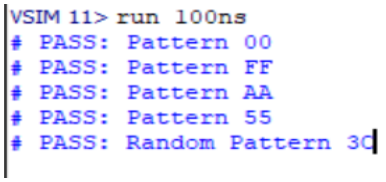
Successful verification of all 16 memory locations in Dual-Port RAM

5. Verification Requirements

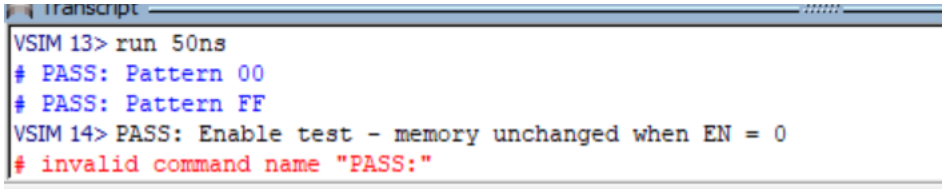
- Verify all 16 addresses: 0 through 15.
- Verify all-zero, all-one, alternating (AA/55), and random data patterns.
- For single-port RAM, verify write followed by readback for every address.
- Verify enable-disabled cycles do not modify memory.
- For dual-port RAM, verify independent simultaneous accesses to different addresses.
- Verify Port A and Port B can both read simultaneously.
- Verify Port A and Port B can both write different addresses simultaneously.
- Explicitly test same-address read/write and same-address write/write collision behavior.
- Check that address values outside the 4-bit range cannot occur.
- Check synchronous read latency according to the chosen implementation.



screenshot of testbench / verification results



Single-Port RAM data pattern verification



To check the enable

```
VSIM 28> run 50ns
# TEST 1: Same-address WRITE/WRITE collision
# Port A writes AA to address 6
# Port B writes 55 to address 6
# RESULT: Write/Write collision detected - result is undefined
# TEST 2: Same-address WRITE/READ
# Port A writes CC to address 8
# Port B reads address 8
# RESULT: Read-during-write is implementation-defined
# Port B read value = cc
# ** Note: $stop    : D:/tb_dual_port_collision.v(100)
#    Time: 36 ns  Iteration: 0  Instance: /tb_dual_port_collision
# Break in Module tb_dual_port_collision at D:/tb_dual_port_collision.v line 100
```

Verification of Dual-Port RAM collision conditions

6. Suggested RTL Requirements

- Use a synthesizable memory array: logic [7:0] mem [0:15];
- Use nonblocking assignments for sequential memory operations.
- Do not initialize or reset every memory location unless explicitly required.
- For dual-port RAM, keep Port A and Port B control logic independent while sharing the memory array.
- Document the exact collision semantics before writing the reference model and scoreboard.

```
Ln#
 1  module single_port_ram (
 2      input clk,
 3      input rst,
 4      input en,
 5      input we,
 6      input [3:0] addr,
 7      input [7:0] wdata,
 8      output reg [7:0] rdata
 9  );
10
11      reg [7:0] mem [0:15];
12
13      always @(posedge clk) begin
14
15          if (en) begin
16
17              if (we) begin
18                  mem[addr] <= wdata;
19              end
20
21              else begin
22                  rdata <= mem[addr];
23              end
24
25          end
26
27      end
28
29  endmodule
```

Single-Port RAM Verilog RTL code

```
Ln#
 1  module dual_port_ram (
 2      // Port A
 3      input clk_a,
 4      input en_a,
 5      input we_a,
 6      input [3:0] addr_a,
 7      input [7:0] wdata_a,
 8      output reg [7:0] rdata_a,
 9
10      // Port B
11      input clk_b,
12      input en_b,
13      input we_b,
14      input [3:0] addr_b,
15      input [7:0] wdata_b,
16      output reg [7:0] rdata_b
17  );
18
19      // Shared 16 x 8 memory
20      reg [7:0] mem [0:15];
21
22      // Port A
23      always @(posedge clk_a) begin
24          if (en_a) begin
25              if (we_a)
26                  mem[addr_a] <= wdata_a;
27              else
28                  rdata_a <= mem[addr_a];
29          end
30      end
31
32      // Port B
33      always @(posedge clk_b) begin
34          if (en_b) begin
35              if (we_b)
36                  mem[addr_b] <= wdata_b;
```

```
29          end
30      end
31
32      // Port B
33      always @(posedge clk_b) begin
34          if (en_b) begin
35              if (we_b)
36                  mem[addr_b] <= wdata_b;
37              else
38                  rdata_b <= mem[addr_b];
39          end
40      end
41
42  endmodule
```

Dual-Port RAM Verilog RTL code

7. Results / Observation

The expected design behavior is determined by the supplied specification. A completed implementation should demonstrate correct storage and retrieval across all 16 addresses, correct synchronous operation, and independent dual-port access where applicable. Collision behavior should be verified against the implementation-defined rules selected for the design

8. Conclusion

The 16 × 8 RAM specification defines a compact synchronous memory with 16 addressable locations and 8-bit data words. The single-port configuration supports one operation per cycle, while the dual-port

configuration allows independent access through two ports. Verification should cover every address, multiple data patterns, simultaneous operations, synchronous read timing, and collision cases.

9. Source Note

This report is based on the uploaded document titled “RAM 16 × 8 Design Specification – Single-Port and Dual-Port Synchronous RAM.” The source notes that it is a learning-oriented RTL specification and that actual FPGA/ASIC projects should finalize latency, reset semantics, byte enables, clock-domain behavior, and collision behavior against the target memory technology.